PHASE 2 | TECHNICAL REPORT

AI-based Village Pond Planning System

Contour-map backend API for terrain-derived pond-site and catchment estimation

SUBMISSION STATUS	LOCAL VALIDATION
Backend implementation complete	**4 automated tests passed**
KML and KMZ upload support	Sample map analyzed successfully

Repository URL

https://github.com/Galabavamsi/ai-village-pond-planning

Working API URL

Local: http://127.0.0.1:8000/analyzeContour

Remote: http://10.1.75.53:8000/analyzeContour

Public: [deployed public API URL - add after reverse proxy]

1. Executive summary

This phase delivers a backend API that accepts a contour map in KML or KMZ format and derives terrain-based pond planning information. The service parses contour elevations, builds a local metric elevation grid, routes surface flow using D8 downhill routing, and returns ranked pond candidates with catchment area and GeoJSON geometries.

> The implementation is generalized: no sample-specific coordinates, locations, or result values are embedded in the code. The supplied contour map is used only as a test input.

Phase 2 objectives

1. Upload	2. Parse	3. Analyze	4. Return
KML / KMZ file	Contour geometry + elevations	Metric grid + D8 flow	JSON + GeoJSON recommendations

The API is intentionally structured as an independent analysis service so later phases can add rainfall, government-land availability, soil, satellite imagery, and a frontend map without replacing the core route.

2. Requirements traceability

Requirement	Implementation evidence	Status
Backend route	POST /analyzeContour; compatibility alias /findCatchment.	Complete
Upload contour map	Multipart upload field file; supports .kml and .kmz.	Complete
Analyze terrain	Contour parsing, interpolation to an elevation grid, and D8 flow routing.	Complete
Identify pond region	Ranks interior cells with high terrain-derived contributing flow and returns a pond footprint.	Complete
Estimate catchment	Upstream contributing cells are merged into a GeoJSON Polygon or MultiPolygon with area metrics.	Complete
No hard-coding	Coordinates and recommendations are calculated from each uploaded input.	Complete
Documentation/demo	README, this report, OpenAPI docs, automated tests, and sample-map results.	Complete

The public GitHub and deployed API fields on the cover must be filled after the remote repository and hosting setup are completed.

3. Catchment estimation methodology

Step	Operation	Output
1	Parse KML/KMZ	Contour lines and elevation values
2	Convert coordinates	Local equirectangular metre-based coordinates
3	Interpolate terrain	Regular elevation grid

Step	Operation	Output
4	Route flow	Each cell flows to its steepest lower neighbour
5	Accumulate area	Upstream cell count and square-metre area
6	Rank candidates	Interior high-contributing pond regions
7	Export geometry	WGS84 GeoJSON location, pond region, and catchment

Coordinate handling

Input coordinates are assumed to be standard WGS84 longitude/latitude as normally stored in KML. The analysis temporarily uses a local equirectangular approximation centered on the input extent so cell areas and flow distances are measured in metres. Returned geometries are converted back to WGS84 GeoJSON for direct map rendering.

Design rationale

A D8 flow model is transparent, reproducible, and extensible for this phase. It gives the frontend a meaningful catchment boundary while keeping later additions independent: rainfall can become a runoff-weighting layer, land ownership can become a constraint mask, and detailed hydrology can replace or refine the routing engine.

4. API documentation

Endpoint

```
POST /analyzeContour
Content-Type: multipart/form-data
Field: file=<contour-map.kml|contour-map.kmz>
```

Query parameters

Parameter	Type / default	Description
grid_size	integer / 100	Cells along the longer map dimension; range 40-220.
max_candidates	integer / 3	Number of ranked pond recommendations; range 1-5.

Success response

The JSON response contains analysis status, input contour statistics, terrain-grid metadata, and a list of recommendations. Each recommendation includes a WGS84 point, pond region, elevation, depth/storage estimate, and a catchment object with area in square metres/hectares and GeoJSON geometry.


```
{
"analysis": {"status": "completed", "algorithm_version": "terrain-d8-v1"},
"input": {"contour_features": 2710, "contour_interval_m": 1.0},
"recommendations": [{
"site_id": "pond-1",
"location": {"type": "Point", "coordinates": [81.2899372847, 21.2499875606]},
"catchment": {"area_hectares": 16.0327, "geometry": {"type": "MultiPolygon"}}
}]
}
```


Additional routes

GET /health returns service status. GET /docs opens the interactive Swagger UI. POST /findCatchment is a compatibility alias for the main analysis route.

5. Demonstration using the supplied contour map

Test input: contour-maps/contours_1m.kml (approximately 6.7 MB). The endpoint was exercised locally through HTTP and returned status 200.

Observed metric	Result
Contour features	2,710
Elevation range	267-298 m
Contour interval	1 m
First recommended point	81.2899372847, 21.2499875606
First estimated catchment	16.0327 hectares
First storage estimate	4,233.98 m3

Reproduce locally

```
python run.py

# In another PowerShell window
curl.exe -X POST "http://127.0.0.1:8000/analyzeContour?grid_size=100&max_candidates=3" `
-F "file=@contour-maps/contours_1m.kml"
```

The interactive alternative is http://127.0.0.1:8000/docs: expand POST /analyzeContour, select Try it out, upload the KML, and execute the request.

6. Interactive Swagger UI evidence

The following screenshots document the complete local demonstration flow. They show the available routes, the selected sample file, and the successful HTTP 200 response returned by the analysis endpoint.

Usage sequence

Start the service with python run.py, open http://127.0.0.1:8000/docs, expand POST /analyzeContour, choose Try it out, upload contours_1m.kml, and press Execute. The response appears under Server response with status 200.

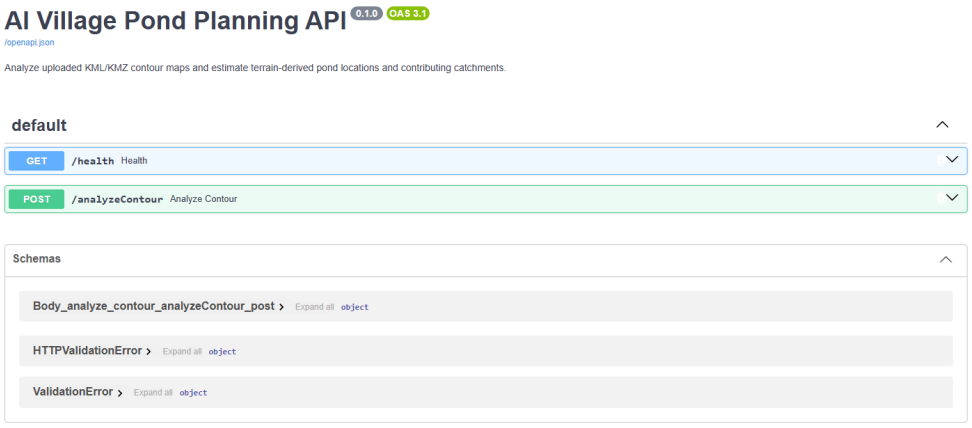

Figure 1. Swagger UI overview showing the health check and contour-analysis routes.

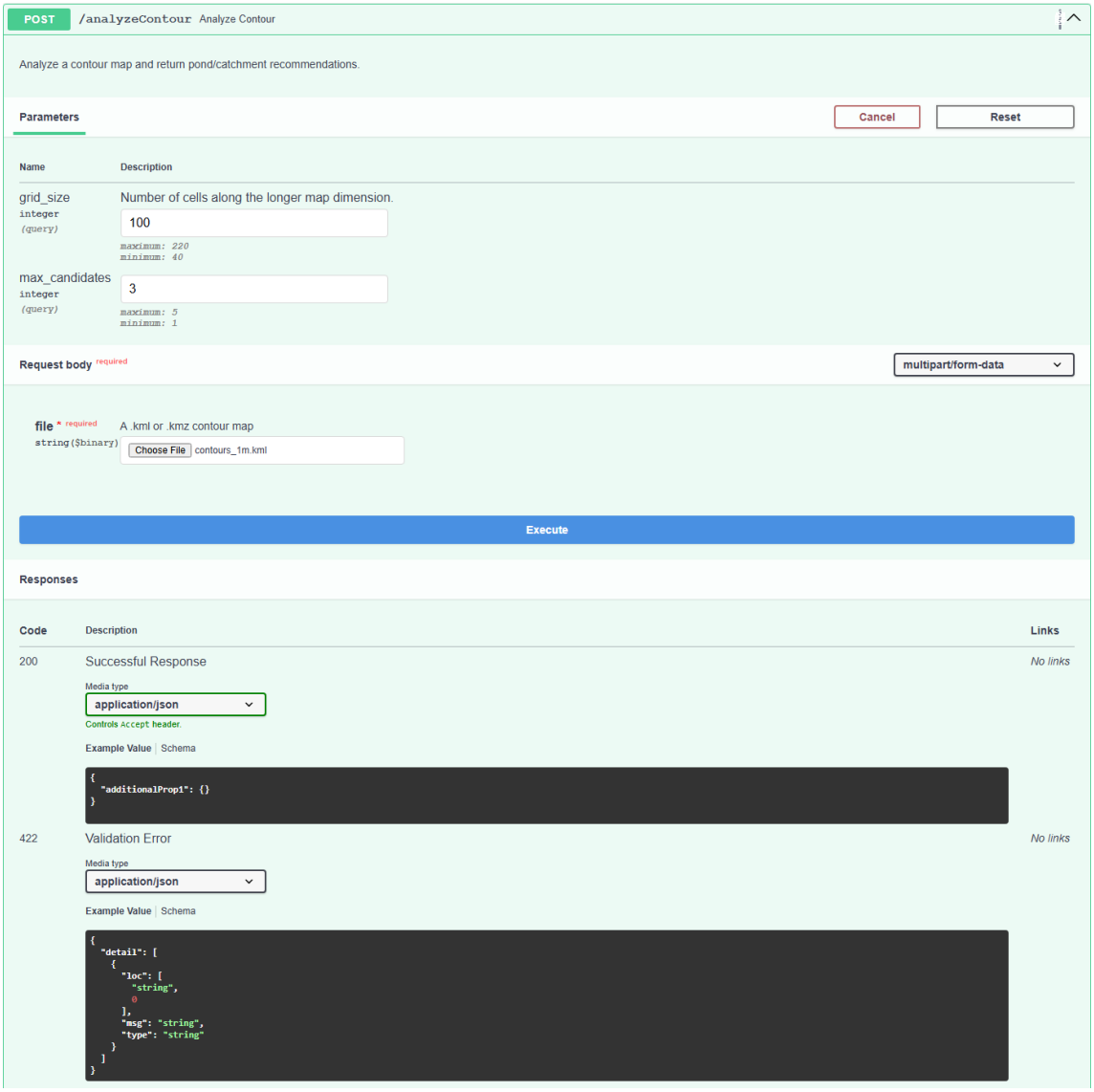

Figure 2. Expanded POST /analyzeContour form with the sample KML selected and Execute button ready.

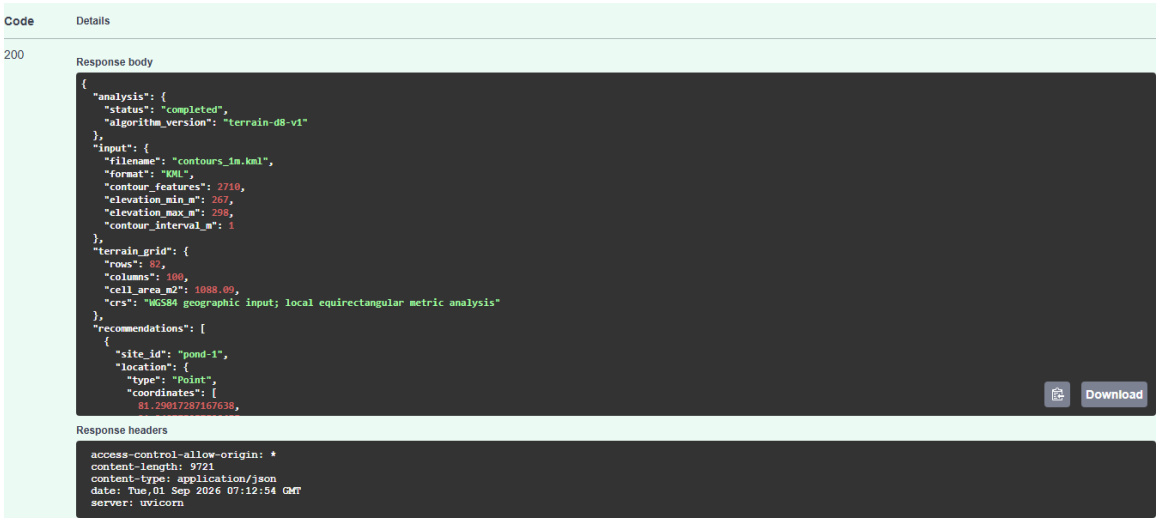

Figure 3. Successful HTTP 200 response showing completed analysis, contour statistics, terrain grid, and recommendation JSON.

The response can be consumed directly by a frontend map because pond and catchment geometries are returned as WGS84 GeoJSON.

7. Validation and deployment

Automated validation

Test	Purpose	Result
Health route	Confirms service responds	Passed
Sample KML analysis	Confirms full terrain pipeline	Passed
Unsupported extension	Confirms upload validation	Passed
Sample KMZ analysis	Confirms archive extraction	Passed

Remote deployment

```
git clone https://github.com/Galabavamsi/ai-village-pond-planning.git
cd ai-village-pond-planning
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
tmux new -s pond-api
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

After starting Uvicorn, detach from tmux with Ctrl-b then d. Reattach with tmux attach -t pond-api. A reverse proxy or secure tunnel should expose the service using the remote host's public URL. SSH credentials and tokens must remain outside the repository.

8. Limitations and future phases

The current result is a terrain-only planning estimate, not a final civil-engineering design. The reported storage estimate uses a conservative compact footprint and depth approximation derived from contour interval. It does not yet model rainfall intensity, runoff coefficient, infiltration, evaporation, land ownership, soil, structures, or field constraints.

Future layer	Planned extension
Rainfall	Apply rainfall and runoff coefficients to convert catchment area into seasonal volume scenarios.
Government land	Intersect candidate pond/catchment geometries with land-ownership boundaries.
Satellite data	Add land cover, drainage, vegetation, and water-body constraints.
Hydraulic design	Replace the approximate footprint/depth estimate with a stage-area-volume model.
Frontend map	Render returned GeoJSON on an interactive map and expose analysis parameters.